Absolutely. If your goal is to become a **job-ready AI Engineer / GenAI Engineer**, you need more than just Python and ML. Think of the skill set as a ladder: **programming → data → ML → deep learning → GenAI/LLMs → AI applications → deployment → engineering → system design**.

Since you're already working through Python/OOP and your Employee Management System, you're on the right starting layer.

🧠 Complete AI Engineer Skill & Knowledge Roadmap

1. 🐍 Python — MUST MASTER

You should be comfortable writing Python without depending heavily on tutorials.

Core Python

- Variables & data types
- Operators
- Strings
- Lists
- Tuples
- Sets
- Dictionaries
- `if/elif/else`
- `for` / `while`
- Functions
- Parameters & arguments
- `*args` / `**kwargs`
- Lambda functions
- `map()`, `filter()`, `reduce()`
- List/dict/set comprehensions
- Exception handling
- File handling
- Modules & packages
- Virtual environments
- `pip`
- Requirements files

Advanced Python

- OOP
  - Classes
  - Objects
  - Inheritance
  - Encapsulation
  - Polymorphism
  - Abstraction
- Iterators
- Generators
- Decorators
- Context managers
- Type hints
- Dataclasses
- Abstract classes
- Magic/dunder methods
- Logging
- Testing with `pytest`
- Debugging
- Async programming basics

You should be able to build

- CLI applications
- Data-processing scripts
- APIs
- Automation scripts
- AI applications

---

2. 🧮 Mathematics for AI

You don't need PhD-level mathematics, but you need enough to understand **why models work**.

Linear Algebra

- Scalars
- Vectors
- Matrices
- Matrix multiplication
- Dot product

- Transpose
- Norms
- Eigenvalues/eigenvectors
- Matrix decomposition
- Embeddings

Calculus

- Functions
- Derivatives
- Partial derivatives
- Gradients
- Chain rule
- Gradient descent

Probability

- Probability basics
- Conditional probability
- Bayes theorem
- Random variables
- Probability distributions
- Expectation
- Variance
- Covariance

Statistics

- Mean
- Median
- Mode
- Standard deviation
- Correlation
- Sampling
- Hypothesis testing
- Confidence intervals
- Outliers

Optimization

- Gradient descent
- Learning rate
- Loss functions
- Regularization

- Convex/non-convex optimization basics

---

3. 📊 NumPy

You should understand numerical computing rather than treating NumPy as just another library.

Learn:

- Arrays
- Dimensions/axes
- Indexing
- Slicing
- Broadcasting
- Vectorization
- Reshaping
- Aggregation
- Matrix operations
- Random numbers
- Linear algebra
- Performance considerations

---

4. 🐼 Pandas

Important for almost every AI/ML role.

Learn:

- Series/DataFrame
- Loading CSV/Excel/JSON
- Selecting/filtering
- Missing values
- Duplicates
- GroupBy
- Merge/join
- Concatenation
- Pivot tables
- Sorting
- Aggregation

- Datetime
- Categorical data
- Data cleaning
- Feature creation

You should be able to take a **messy dataset → clean dataset → ML-ready dataset**.

---

5. 📈 Data Visualization

Matplotlib

- Line plots
- Bar charts
- Histograms
- Scatter plots
- Box plots
- Subplots
- Customization

Seaborn

- Distribution plots
- Correlation heatmaps
- Categorical plots
- Statistical visualization

Also understand:

- What visualization to use for which problem
- How to identify patterns
- How to detect outliers
- How to communicate insights

---

6. 🗄️ SQL — VERY IMPORTANT

Many aspiring AI engineers underestimate SQL.

Learn:

Basics

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING
- DISTINCT
- LIMIT

Intermediate

- JOINs
- Subqueries
- CTEs
- CASE
- Window functions
- Aggregations

Advanced

- Query optimization
- Indexes
- Transactions
- Database design
- Normalization

Databases:

- PostgreSQL
- MySQL
- SQLite

You don't necessarily need all three. **PostgreSQL is an excellent one to focus on.**

---

7. 🤖 Machine Learning

This is where you start becoming an AI engineer.

Supervised Learning

Regression

- Linear Regression
- Polynomial Regression
- Ridge
- Lasso
- Elastic Net

Classification

- Logistic Regression
- KNN
- Naive Bayes
- Decision Trees
- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost
- SVM

---

8. 🔍 Unsupervised Learning

Learn:

- K-Means
- Hierarchical clustering
- DBSCAN
- PCA
- Dimensionality reduction
- Anomaly detection

Understand:

- When clustering is useful
- Choosing number of clusters
- Cluster evaluation

---

9. 🧪 ML Evaluation

This is **extremely important**.

Learn:

Regression

- MAE
- MSE
- RMSE
- R^2
- MAPE

Classification

- Accuracy
- Precision
- Recall
- F1
- Confusion matrix
- ROC-AUC
- PR-AUC

Also:

- Cross-validation
- Train/validation/test split
- Stratification
- Data leakage
- Overfitting
- Underfitting
- Bias/variance tradeoff

---

10. 🛠️ Scikit-learn

You should be able to build an ML project using:

Data
↓
EDA

↓
Cleaning
↓
Feature Engineering
↓
Train/Test Split
↓
Preprocessing
↓
Model
↓
Evaluation
↓
Hyperparameter Tuning
↓
Save Model
↓
API

Learn:

- Pipelines
- ColumnTransformer
- Feature preprocessing
- Model selection
- GridSearchCV
- RandomizedSearchCV
- Model persistence

---

11. 🧠 Deep Learning

Now move into neural networks.

Fundamentals

- Perceptron
- Neural networks
- Neurons
- Activation functions
- Forward propagation
- Backpropagation

- Loss functions
- Optimizers
- Learning rate
- Batch size
- Epochs
- Regularization
- Dropout
- Batch normalization

Activation functions

- ReLU
- Sigmoid
- Tanh
- Softmax
- GELU

Optimizers

- SGD
- Momentum
- Adam
- AdamW
- RMSProp

---

12. 🔥 PyTorch

For an AI Engineer, I strongly recommend becoming comfortable with **PyTorch**.

Learn:

- Tensors
- Dataset
- DataLoader
- `nn.Module`
- Layers
- Forward pass
- Loss functions
- Optimizers
- Training loops
- Validation loops

- GPU/CUDA
- Model saving/loading
- Transfer learning
- Fine-tuning

Also understand how to debug things like:

loss = NaN
accuracy = 0
gradient explosion
gradient vanishing
CUDA errors
shape mismatch

You've already encountered the `loss = nan` problem, which is actually useful experience.

---

13. 👁️ Computer Vision

If you want broader AI-engineering capability:

Fundamentals

- Images as tensors
- Convolution
- Kernels
- Pooling
- CNNs
- Image augmentation

Models

- ResNet
- EfficientNet
- YOLO
- Vision Transformers

Tasks

- Image classification
- Object detection
- Image segmentation

- OCR
- Face/feature embeddings

Libraries:

- OpenCV
- PyTorch
- torchvision

---

14. 📝 NLP

Understand traditional NLP before jumping completely into LLMs.

Learn:

- Tokenization
- Stopwords
- Stemming
- Lemmatization
- N-grams
- TF-IDF
- Bag of Words
- Word embeddings
- Word2Vec
- GloVe
- Named Entity Recognition
- Text classification
- Sentiment analysis

---

15. 🤯 Transformers — CRITICAL for GenAI

This is one of the most important areas for your target.

Understand:

Transformer architecture

Text
↓
Tokenization
↓
Embeddings
↓
Positional Encoding
↓
Self Attention
↓
Multi-Head Attention
↓
Feed Forward Network
↓
Layer Normalization
↓
Transformer Blocks
↓
Output

Learn:

- Attention
- Self-attention
- Query
- Key
- Value
- Multi-head attention
- Positional encoding
- Encoder
- Decoder
- Encoder-decoder architecture
- Causal attention
- KV cache
- Context window

---

16. 🤖 LLMs

Understand how modern LLMs work.

Learn:

- GPT-style models
- BERT-style models
- Llama-family models
- Mistral-family models
- Tokenizers
- Embeddings
- Context windows
- Temperature
- Top-k
- Top-p
- Prompt tokens
- Output tokens
- Inference

You should understand the difference between:

training vs fine-tuning vs inference vs RAG.

---

17. ✍️ Prompt Engineering

Learn:

- Zero-shot prompting
- Few-shot prompting
- Role prompting
- Structured prompting
- Chain-of-thought concepts
- Output constraints
- JSON output
- Function/tool calling
- Prompt templates
- Prompt evaluation

More importantly, learn **when prompting is NOT the right solution**.

---

18. 🔎 RAG — VERY IMPORTANT

RAG = Retrieval-Augmented Generation.

Understand:

Documents
↓
Chunking
↓
Embedding
↓
Vector Database
↓
Similarity Search
↓
Retrieved Context
↓
LLM
↓
Answer

Learn:

- Document loaders
- Chunking
- Embeddings
- Vector search
- Metadata filtering
- Hybrid search
- Reranking
- Context compression
- Retrieval evaluation
- Hallucination reduction

---

19. 🧠 Vector Databases

Learn at least one deeply.

Examples:

- FAISS
- Chroma
- Qdrant
- Pinecone
- Weaviate

- Milvus

Understand:

- Vector embeddings
- Similarity search
- Cosine similarity
- Approximate nearest neighbor search
- Metadata filtering
- Indexing

---

20. 🔗 AI Agent Development

Modern AI engineering increasingly involves tool-using systems.

Learn:

- Agents
- Tools
- Function calling
- Planning
- Memory
- Tool selection
- Multi-step workflows
- Agent loops
- Human-in-the-loop
- Guardrails

Frameworks worth knowing:

- LangChain
- LangGraph
- LlamaIndex

Don't just learn frameworks. Understand the **underlying architecture**.

---

21. 🔧 AI APIs

Learn how to integrate model APIs.

Understand:

- API authentication
- REST APIs
- JSON
- Requests
- Streaming
- Rate limits
- Retries
- Timeouts
- Structured outputs
- Tool/function calling
- Error handling

Also learn to work with multiple model providers rather than becoming dependent on one API.

---

22. 🌐 Backend Development

An AI engineer needs to turn models into applications.

Learn:

FastAPI

- Routes
- GET/POST
- Request models
- Response models
- Validation
- Authentication basics
- Async endpoints
- Error handling
- API documentation

Also understand:

- REST
- HTTP
- JSON
- WebSockets basics

---

23. 🖥️ Frontend Basics

You don't need to become a frontend engineer.

But learn enough to build AI interfaces.

HTML

CSS

JavaScript basics

And preferably:

- Streamlit
- Gradio

These are especially useful for AI prototypes.

---

24. 🐳 Docker

You should know how to package your AI application.

Learn:

- Dockerfile
- Images
- Containers
- Volumes
- Networks
- Environment variables
- Docker Compose
- Containerizing FastAPI
- Containerizing ML applications

---

25. ☁️ Cloud

Know at least **one cloud platform**.

Choose one:

- AWS
- Azure
- Google Cloud

For example, AWS:

- EC2
- S3
- IAM
- Lambda
- CloudWatch
- ECR
- ECS basics

For AI specifically, understand:

- GPU instances
- Model deployment
- Storage
- Networking
- Secrets
- Monitoring

---

26. 🚀 MLOps

This separates someone who can build a model from someone who can operate AI systems.

Learn:

- Experiment tracking
- Model versioning
- Data versioning
- Model registry
- CI/CD
- Automated testing
- Deployment
- Monitoring
- Logging

- Model drift
- Data drift
- Retraining

Tools:

- MLflow
- DVC
- GitHub Actions
- Docker
- Kubernetes basics

---

27. 🧪 LLMOps

For GenAI jobs, learn:

- Prompt versioning
- LLM evaluation
- Tracing
- Token monitoring
- Cost monitoring
- Latency monitoring
- Hallucination evaluation
- RAG evaluation
- Agent evaluation
- Guardrails
- Production logging

Understand metrics such as:

- Latency
- Throughput
- Tokens/request
- Cost/request
- Retrieval precision
- Answer faithfulness

---

28. 🗃️ Databases

Know both:

Relational

- PostgreSQL
- MySQL

NoSQL

- MongoDB
- Redis

Redis is particularly useful for:

- Caching
- Sessions
- Rate limiting
- Fast data access

---

29. 🐙 Git & GitHub

Non-negotiable for jobs.

Learn:

git init
git clone
git status
git add
git commit
git push
git pull
git branch
git merge
git rebase

Also:

- Pull requests
- Issues
- `.gitignore`
- README files

- GitHub Actions
- Branching strategies

---

30. 🐧 Linux

Learn basic Linux commands:

ls
cd
pwd
mkdir
rm
cp
mv
cat
grep
find
chmod
ps
kill
top

Also:

- Environment variables
- Processes
- SSH
- Shell scripting basics

---

31. 🏗️ Software Engineering

This is often overlooked.

Learn:

- Clean code
- SOLID principles
- Design patterns

- Modular architecture
- Code reviews
- Unit testing
- Integration testing
- Logging
- Error handling
- Configuration management
- Secrets management
- Documentation

---

32. 🧩 System Design

For higher-level AI engineering roles, learn how to design systems like:

AI chatbot

User
↓
Frontend
↓
API
↓
Authentication
↓
RAG/Agent
↓
Vector DB
↓
LLM
↓
Response

Understand:

- Scalability
- Load balancing
- Caching
- Queues
- Databases
- Rate limiting
- Async processing
- Fault tolerance

- Monitoring

---

33. 🔐 AI Security

Very important for production AI.

Learn:

- Prompt injection
- Jailbreaking
- Data leakage
- PII protection
- Authentication
- Authorization
- API security
- Secrets management
- Input validation
- Output validation
- Rate limiting

For RAG:

- Access-controlled retrieval
- Document permissions
- Data isolation

---

34. ⚖️ Responsible AI

Understand:

- Bias
- Fairness
- Explainability
- Privacy
- Safety
- Data governance
- Model limitations
- Hallucinations

---

35. 📚 Research Skills

You don't need to become an AI researcher, but you should be able to read papers.

Learn:

- How to read an ML paper
- Abstract
- Methodology
- Experiments
- Baselines
- Ablation studies
- Evaluation metrics

Important resources include papers on:

- Transformers
- BERT
- GPT
- RAG
- LoRA
- RLHF
- Diffusion models

---

36. 🔬 Fine-Tuning

Eventually learn:

- Transfer learning
- Full fine-tuning
- LoRA
- QLoRA
- PEFT
- Instruction tuning
- Supervised fine-tuning
- Dataset preparation
- Evaluation

Understand when:

Prompting → RAG → Fine-tuning

is appropriate.

---

37. 🎨 Generative AI Beyond Text

Optional but valuable.

Image generation

- Diffusion models
- Stable Diffusion concepts
- Image embeddings
- Multimodal models

Audio

- Speech-to-text
- Text-to-speech
- Audio embeddings

Multimodal AI

- Text + image
- Text + audio
- Vision-language models

---

38. 📦 AI Engineering Libraries

You don't need to memorize every library.

Your toolbox should eventually include:

Python
NumPy
Pandas

Matplotlib
Seaborn
Scikit-learn
PyTorch
Transformers
OpenCV
FastAPI
Pydantic
LangChain
LangGraph
LlamaIndex
FAISS/Qdrant
MLflow
Docker
Git
PostgreSQL
Redis

---

39. 💼 Business & Problem-Solving Skills

This is surprisingly important.

An AI engineer needs to answer:

> "Why are we using AI here?"

Learn:

- Problem formulation
- Requirements gathering
- Cost-benefit analysis
- Model selection
- Trade-offs
- Latency vs accuracy
- Cost vs quality
- Build vs buy
- Evaluation criteria
- Communicating technical concepts

---

40. 🧑‍💻 Portfolio Projects

This is where everything comes together.

You should eventually have **6–10 serious projects**, not 30 tiny tutorials.

Project 1 — Classical ML

Customer Churn Prediction

Python + Pandas + Scikit-learn + FastAPI

Project 2 — Deep Learning

Image Classification System

PyTorch + CNN + transfer learning

Project 3 — NLP

Document Classification / Sentiment Analysis

Transformers + Hugging Face

Project 4 — RAG

Personal Knowledge AI

PDF
↓
Chunking
↓
Embeddings
↓
Vector DB
↓
Retriever
↓
LLM

↓
Answer

Project 5 — Advanced RAG

Build:

Enterprise Document Assistant

with:

- Authentication
- Document ingestion
- Metadata
- Hybrid retrieval
- Reranking
- Citations
- Evaluation
- FastAPI
- PostgreSQL
- Vector DB
- Docker

Project 6 — AI Agent

Build an agent capable of:

User
↓
LLM
↓
Choose tool
↓
Execute tool
↓
Observe result
↓
Reason
↓
Next tool
↓
Final answer

Project 7 — Production AI

Deploy an AI application using:

Frontend
↓
FastAPI
↓
AI Service
↓
Vector DB
↓
LLM
↓
PostgreSQL
↓
Docker
↓
Cloud

This is the type of project that demonstrates **AI engineering**, rather than just model training.

🎯 What YOU should learn in order

Given your current progress, I would **not** recommend trying to learn everything simultaneously.

Your path should be:

PHASE 1
Python
↓
OOP
↓
Git/GitHub
↓
NumPy
↓
Pandas
↓
Matplotlib/Seaborn

↓
SQL

Then:

PHASE 2
Statistics
↓
Linear Algebra
↓
Machine Learning
↓
Scikit-learn
↓
ML Projects

Then:

PHASE 3
Deep Learning
↓
PyTorch
↓
CNN
↓
NLP
↓
Transformers

Then:

PHASE 4
LLMs
↓
Prompt Engineering
↓
Embeddings
↓
Vector Databases
↓
RAG
↓
Fine-tuning
↓
Agents

Then:

PHASE 5
FastAPI
↓
Docker
↓
Cloud
↓
MLOps
↓
LLMOps
↓
System Design

Finally:

AI ENGINEER
↓
Build
↓
Evaluate
↓
Deploy
↓
Monitor
↓
Improve

⭐ The most important skills for your target

If I had to compress the entire roadmap into the **core 20**, I'd prioritize:

Priority	Skill
🔴 1	Python
🔴 2	OOP
🔴 3	Git/GitHub
🔴 4	NumPy
🔴 5	Pandas

🔴 6 SQL

🔴 7 Statistics

🔴 8 Machine Learning

🔴 9 Scikit-learn

🔴 10 Deep Learning

🔴 11 PyTorch

🔴 12 Transformers

🔴 13 LLMs

🔴 14 RAG

🔴 15 Vector Databases

🔴 16 AI Agents

🔴 17 FastAPI

🔴 18 Docker

🔴 19 Cloud

🔴 20 MLOps/LLMOps

One important point: you do **not** need to master all 40 areas before applying for an internship. The goal should be to become **strong in the core stack first**, then progressively add production and advanced GenAI skills.

For you specifically, I would continue from your current **Day 9 OOP → NumPy/Pandas → ML → Deep Learning → GenAI**, rather than restarting from zero.